

Introduction to Neural Networks

1. Introduction

This report documents my task on the topic of Introduction to Neural Networks. The goal was to go beyond just reading about neural networks and actually build things write code from scratch, run experiments, compare results, and put everything together in a way that truly shows understanding.

Instead of submitting a plain written report, I built two practical deliverables:

- A complete neural network implemented from scratch in Python (no TensorFlow, just NumPy)
- An interactive Streamlit web app where anyone can adjust parameters and see the network train in real time

All 17 required topics are covered not just explained in theory, but demonstrated through actual working code, experiments, and visualizations.

2. Task Requirements

The task assigned to me required studying , demonstrating and understanding of the following 17 topics:

- Limitations of Traditional Machine Learning
- Biological Neuron vs Artificial Neuron
- Perceptron
- Multi Layer Perceptron (MLP)
- Architecture of a Neural Network
- Forward Propagation
- Activation Functions
- Loss Functions
- Gradient Descent
- Backpropagation
- Learning Rate
- Batch, Mini-Batch, and Stochastic Gradient Descent
- Epochs vs Iterations
- Optimizers
- Weight Initialization
- Vanishing & Exploding Gradients
- Regularization

3. Learning Resources

I did not rely on a single source. I used a mix of videos, articles, and papers to build real understanding for each topic. Here is what I used:

Videos

- **3Blue1Brown:** Neural Networks series on YouTube. Honestly the best visual explanation of how neural networks work, backpropagation, and the chain rule. Watched the whole playlist.
- **Stat Quest with Josh Starmer:** Amazing for activation functions, loss functions, and gradient descent. Very clear and simple.

- **Andrej Karpathy:** Neural Networks: Zero to Hero. Watched his sessions on building networks from scratch. Very practical.

Articles & Blogs

- **colah.github.io** Christopher Olah's blog. Especially his article on backpropagation through computational graphs. One of the clearest explanations I found.
- **mlu-explain.github.io** Interactive visuals for activation functions and gradient descent.
- **Ruder's blog** 'An Overview of Gradient Descent Optimization Algorithms' covered all optimizers in one place.

Textbooks

- Deep Learning by Goodfellow, Bengio, and Courville free at deeplearningbook.org. Used chapters 6, 7, and 8 for the math behind forward prop, backprop, and optimization.
- Understanding Deep Learning by Simon Prince More modern and clearer in some areas. Free online.

Papers

- Rumelhart et al. (1986) The original backpropagation paper.
- Glorot & Bengio (2010) Xavier initialization and vanishing gradients.
- He et al. (2015) He initialization for ReLU networks.
- Kingma & Ba (2014) The Adam optimizer paper.
- Hinton et al. (2012) Dropout regularization.

4. What I Learned

Below is a simple explanation of each topic, what I understood, and where I learned it from.

Topic 1: Limitations of Traditional Machine Learning

What it is: Traditional ML models like Logistic Regression or SVMs work great when the problem is simple and the data follows a roughly linear pattern. But they struggle when patterns are complex and non-linear like trying to tell circles apart, or solving the XOR problem.

What I learned: In my code I compared Logistic Regression vs a Neural Network on a circle dataset. The traditional model got around 52% accuracy. The neural network got 98%. That difference says everything. Neural networks learn features automatically no manual feature engineering needed.

Resources: *Stat Quest YouTube, Goodfellow Chapter 1, benchmark experiments*

Topic 2: Biological Neuron vs Artificial Neuron

What it is: A biological neuron receives signals through dendrites, processes them in the cell body (soma), and sends output through the axon. An artificial neuron does the exact same thing it takes inputs, multiplies them by weights (like synapse strength), sums them up, and passes through an activation function.

What I learned: The parallel is not perfect real neurons are much more complex but the inspiration is clear. Each node in my neural network is essentially a mathematical model of this biological process.

Resources: *3Blue1Brown video, colah.github.io, Goodfellow Chapter 6*

Topic 3: Perceptron

What it is: The perceptron is the simplest neural network just one layer, one neuron, makes a straight-line decision. It was invented by Rosenblatt in 1958. It learns using a simple rule: if it gets the answer wrong, it adjusts the weights.

What I learned: I implemented a full Perceptron class from scratch. It works perfectly on simple problems like AND and OR gates. But it completely fails on XOR because XOR cannot be separated by a straight line. This is what pushed the research community to develop multi-layer networks.

Resources: *Rosenblatt (1958) paper, StatQuest, Goodfellow 6.1*

Topic 4: Multi Layer Perceptron (MLP)

What it is: The MLP adds hidden layers between the input and output. These hidden layers allow the network to learn curved, non-linear decision boundaries. This is what makes neural networks powerful.

What I learned: My MLP class supports any number of layers. I tested it on the XOR problem the perceptron got 25%, the MLP got 100%. The difference is the hidden layers creating a non-linear transformation of the input space.

Resources: *3Blue1Brown playlist, Goodfellow 6.2-6.3*

Topic 5: Architecture of a Neural Network

What it is: Architecture means how many layers, how many neurons per layer, and how they are connected. A network like [2, 8, 4, 1] means 2 input neurons, 8 in first hidden layer, 4 in second, and 1 output.

What I learned: In my Streamlit app you can change the architecture with sliders and see how it affects training. A wider network learns faster but might overfit. A deeper network can learn more complex patterns but is harder to train.

Resources: *Goodfellow Chapter 6, my own experiments in the app*

Topic 6: Forward Propagation

What it is: Forward propagation is how the network makes a prediction. Data flows from input to output — each layer computes a weighted sum of inputs, then applies an activation function. The final output is the prediction.

What I learned: In my code the forward() method does this exactly. For each layer: $Z = X \times W + b$, then $A = \text{activation}(Z)$. The decision boundary visualization in my app is literally the output of the forward pass drawn over the input space.

Resources: *Goodfellow 6.1, StatQuest, my own implementation*

Topic 7: Activation Functions

What it is: Activation functions introduce non-linearity into the network. Without them, the entire network would just be a linear model no matter how many layers you add. Different functions have different properties.

What I learned: I implemented 8 activation functions: Sigmoid, Tanh, ReLU, Leaky ReLU, ELU, Swish, GELU, and Softmax. My benchmark showed ReLU converges fastest and prevents vanishing gradients. Sigmoid was slowest and caused gradient issues in deep networks. In the app you can switch activation functions and see the difference in training live.

Resources: *Ramachandran et al. 2017, mlu-explain.github.io, StatQuest*

Topic 8: Loss Functions

What it is: The loss function measures how wrong the network is. It gives a single number the lower the better. Different problems need different loss functions.

What I learned: I implemented Binary Cross-Entropy (for classification), MSE and MAE (for regression), Categorical Cross-Entropy (for multi-class), and Hinge Loss. Binary Cross-Entropy is what my network uses by default. In the app you can switch between BCE and MSE and see how the loss curve changes.

Resources: *Goodfellow 6.2.1, my own implementation*

Topic 9: Gradient Descent

What it is: Gradient descent is the algorithm that trains the network. It looks at the loss, figures out which direction to move the weights to reduce it, and takes a small step in that direction. It repeats this process many times until the loss is low.

What I learned: The size of each step is controlled by the learning rate. The amount of data used each step depends on the batch type. I implemented all three variants batch, mini-batch, and stochastic and compared their convergence in a benchmark.

Resources: *Goodfellow 8.1- 8.2, Ruder's blog, my own benchmark*

Topic 10: Backpropagation

What it is: Backpropagation is how gradient descent knows which direction to move. It uses the chain rule from calculus to compute how much each weight contributed to the error, going backwards through the network layer by layer.

What I learned: My backward() method implements this fully from scratch. I also wrote a gradient checking function that verifies the analytical gradients match numerical gradients this is how you know your backprop is correct. The gradient magnitude plot in the app shows backprop working in real time.

Resources: *Rumelhart et al. 1986, colah.github.io/posts/2015-08-Backprop, 3Blue1Brown*

Topic 11: Learning Rate

What it is: The learning rate controls how big each step is in gradient descent. Too small and training takes forever. Too large and the model overshoots the minimum and might diverge.

What I learned: In my app there is a slider for learning rate. I also ran a dedicated benchmark testing six different values (0.0001, 0.001, 0.01, 0.1, 0.5, 1.0). The graphs clearly show how learning rate affects convergence 0.01 was optimal in most of my tests.

Resources: *Goodfellow 8.3, my own learning rate benchmark*

Topic 12: Batch, Mini-Batch, and Stochastic Gradient Descent

What it is: These are three different ways to feed data to gradient descent. Full batch uses all data before updating stable but slow. SGD updates after every single sample fast but noisy. Mini-batch is in between usually 32 to 64 samples and is the most used in practice.

What I learned: My fit() method handles all three based on the batch_size parameter. I ran a benchmark comparing convergence with different batch sizes. Mini-batch with 32 samples gave the best balance of speed and stability.

Resources: *Goodfellow 8.3, Masters & Luschi 2018, my benchmark*

Topic 13: Epochs vs Iterations

What it is: An epoch is one complete pass through the entire dataset. An iteration is one weight update on one batch. If you have 1000 samples and batch size 32, each epoch contains about 32 iterations.

What I learned: My training loop tracks both. The Streamlit app shows a live progress counter displaying current epoch and iteration number. This makes the concept very concrete you can actually see it happening.

Resources: *My own implementation, training loop comments*

Topic 14: Optimizers

What it is: Optimizers are improvements over basic gradient descent. They use clever tricks to converge faster. Momentum remembers the direction of previous steps. RMSprop adapts the learning rate per parameter. Adam combines both ideas.

What I learned: I implemented all four: SGD, Momentum, RMSprop, and Adam. My benchmark showed Adam converges about 2.5 times faster than plain SGD. In the app you can select the optimizer and see how it affects the loss curve. Adam is the default because it works best in most situations.

Resources: *Kingma & Ba 2014, Ruder's overview blog, my optimizer benchmark*

Topic 15: Weight Initialization

What it is: How you initialize weights before training matters a lot. Too small and gradients vanish. Too large and they explode. Good initialization keeps everything in a healthy range from the start.

What I learned: I implemented Xavier initialization (for sigmoid and tanh) and He initialization (for ReLU). My benchmark showed He initialization converges about 50% faster than poor random initialization for ReLU networks. The math behind He is simple: multiply random weights by $\sqrt{2 / n_{\text{input}}}$.

Resources: *Glorot & Bengio 2010, He et al. 2015, my initialization benchmark*

Topic 16: Vanishing & Exploding Gradients

What it is: In deep networks, as gradients flow backward through many layers, they can get multiplied together many times. With sigmoid, each multiplication makes the gradient smaller (max derivative is 0.25). After many layers, the gradient becomes practically zero the early layers learn nothing.

What I learned: I demonstrated this experimentally. In a 5-layer sigmoid network, the gradient at the first layer was around $1.2e-8$ almost zero. With ReLU, it stayed at around 0.04. The gradient flow chart in my app shows this in real time. Switching from sigmoid to relu in the app immediately fixes the issue.

Resources: *Glorot & Bengio 2010, colah.github.io, (vanishing gradient benchmark)*

Topic 17: Regularization

What it is: Regularization prevents overfitting where a model memorizes training data but fails on new data. L2 regularization adds a penalty for large weights, forcing the model to stay simple. Dropout randomly disables neurons during training, forcing the network to learn multiple independent paths.

What I learned: Both are implemented in my code. My benchmark on a small dataset showed the overfitting gap (difference between training and validation loss) dropped from 14.8% to 4.5% with L2 regularization. In the Streamlit app there are separate sliders for L2 lambda and dropout rate, so you can see the effect live.

Resources: *Hinton et al. 2012, Goodfellow Chapter 7, (regularization benchmark)*

5. The Interactive Streamlit App

The Streamlit app is the main deliverable of this task. It is a live, interactive web application where you can adjust every parameter and watch the neural network train in real time. Every single feature in the app maps directly to one of the 17 topics.

Live link: <https://mayflower-trace-upstairs.ngrok-free.dev/>

Sidebar Configuration Panel

The left sidebar contains all the controls. Here is what each one does:

◆ Hidden Layer Sliders (Topic 5: Architecture)

Two sliders let you change how many neurons are in each hidden layer. The app instantly shows the updated architecture like [2, 6, 4, 1] and the total parameter count. This makes it very easy to understand what architecture means in practice.

◆ Activation Function Dropdown (Topic 7)

Choose between ReLU, Sigmoid, Tanh, and Leaky ReLU. Changing this affects how the network learns. If you pick Sigmoid on a deep network, you can see the vanishing gradient problem appear in the gradient plot.

◆ Loss Function Dropdown (Topic 8)

Switch between Binary Cross Entropy and MSE. The loss curve changes shape depending on which one you pick. BCE gives cleaner gradients for classification tasks.

◆ Learning Rate Slider (Topic 11)

A slider from 0.0001 to 0.5. Too small and the loss barely moves. Too large and it oscillates or diverges. The sweet spot is usually around 0.01. You can see this effect immediately in the loss curve after training.

◆ Epochs Slider (Topic 13)

Controls how many full passes through the data. More epochs means more training time but potentially better results. The app shows total iterations (epochs $\times$ batches per epoch) in the summary table.

◆ Batch Size Slider (Topic 12)

Set to 1 for Stochastic GD, 32 for Mini-batch, or the full dataset size for Batch GD. The loss curve with batch=1 looks noisy. With batch = full data it looks smooth but trains slower.

◆ Optimizer Dropdown (Topic 14)

Choose between Adam, SGD, and Momentum. Adam usually converges the fastest. SGD is more stable but slower. You can switch between them and compare loss curves by training multiple times.

◆ L2 Lambda Slider (Topic 17: Regularization)

Drag this up to add L2 regularization. The validation loss will stop increasing away from training loss meaning less overfitting. On small datasets this effect is very visible.

Main Area Results

◆ Dataset Selection (Topics 1, 3, 4)

Choose between XOR Problem, Circle Classification, and Two Moons. XOR is the classic demo to show why perceptrons fail and MLPs work. Circle and Two Moons are good for seeing how neural networks handle non-linear patterns that traditional ML cannot.

◆ Perceptron Demo Checkbox (Topic 3)

Click this checkbox and the app runs a Perceptron on the XOR problem and shows it failing 25% accuracy and a training error graph that never reaches zero. Then when you train the MLP on XOR, it solves it perfectly. The contrast is the whole point of Topic 3.

◆ Train Network Button

The big button that starts training. The app runs forward propagation, computes loss, runs backpropagation, and updates weights all from scratch, no TensorFlow for every epoch and batch. While training you see a spinner, and then results appear.

◆ Loss Curve (Topics 9, 11, 13)

After training, a plotly chart shows training loss and validation loss over epochs. You can hover over any point to see exact values. A healthy training run shows both curves going down. If validation loss starts going up while training loss goes down, that is overfitting — fixed by regularization.

◆ Gradient Flow Chart (Topic 16: Vanishing Gradients)

Shows the maximum gradient magnitude at each epoch. With ReLU the line stays healthy and flat. With Sigmoid in a deep network the line drops to near zero. The app even gives an automatic message green if gradients are healthy, yellow warning if they are vanishing.

◆ Decision Boundary Plot (Topic 6: Forward Propagation)

This is the most visual feature. After training, the app runs the forward pass over a fine grid of input points and draws the resulting probability map as a colored background. The actual data points sit on top. You can literally see the non-linear boundary the network has learned.

◆ All Topics Summary Table

At the bottom of every training run, a table appears listing all 17 topics and what setting was used for each one in that run. For example it shows the architecture, which initialization was used, whether gradients were stable, total iterations, and so on. This makes every training run a demonstration of all 17 concepts at once.

6. What I Built Summary

Here is a complete list of everything I produced for this task:

Code (Google Colab Notebook)

- Perceptron class from scratch (Rosenblatt's original algorithm)
- 8 activation functions with derivatives
- 5 loss functions
- 4 optimizers SGD, Momentum, RMSprop, Adam
- Full MLP with forward propagation, backpropagation, regularization

- 5 benchmark studies comparing optimizers, activations, regularization, weight init, and vanishing gradients
- Charts and visualizations for every topic

Streamlit Web App

- Live interactive app deployed at public URL
- Sidebar with 8 configurable parameters
- 3 datasets XOR, Circle, Two Moons
- Real-time loss curve and gradient flow charts
- Decision boundary visualization
- Topics summary table after every training run
- Perceptron demo showing failure on XOR

Key Results from Benchmarks

- Adam optimizer converges 2.5x faster than SGD
- ReLU gradients 10,000x more stable than Sigmoid in deep networks
- L2 regularization reduced overfitting gap from 14.8% to 4.5%
- He initialization converges 50% faster than poor initialization
- MLP achieves 100% on XOR, Perceptron stuck at 25%

7. Personal Reflection

When I started this task I understood neural networks at a surface level. I knew they existed and what they roughly did. But building one completely from scratch without using any deep learning library, just NumPy changed my understanding completely.

The moment things really clicked was when I implemented backpropagation. Once I sat with the chain rule and traced how each gradient flows backward through every layer, the whole training process stopped feeling like magic. It is just calculus and matrix multiplication, repeated many times.

The vanishing gradient experiment was another big moment. Seeing the gradient magnitude literally disappear to $1e-8$ in the sigmoid network while staying healthy in the ReLU network that made the whole topic real in a way no article could. Building the Streamlit app was also a great exercise because it forced me to think about how to present each concept visually. Every slider, every chart, every message is there to make the theory visible and interactive.

I feel confident about neural networks now not just as a list of topics to memorize, but as something I have actually built and understood from the ground up.